

DevGuardian

Requirements & Build Flow

Microsoft Hackathon - Theme 06: AI-Powered Production Function: Reinventing Work

Team: The Hallucinato - Muskan Sirse & Yogesh Gurjar

Pipeline modules - REST API - data model - deployment

Updated build - 2026-06-13

1. Pipeline build flow (per PR)

- Module 1 - DTS Engine (app/dts_engine.py): trust lookup + depth routing, explainable breakdown.
- Module 2 - NIM Client (app/nim_client.py): depth-adaptive AI review with a 429 fallback chain.
- Module 3 - DNA Engine (app/dna_engine.py): ChromaDB semantic check (repo-scoped) + trait comparison.
- Module 4 - Security Scanner (app/security_scanner.py): Semgrep OSS, or a built-in OWASP rule pack on Windows; AI explanations run in parallel.
- Module 5 - Test Generator (app/test_generator.py): AST gap detection + AI-generated pytest tests (parallel per file).
- Module 6 - Deploy Gate (app/deploy_gate.py): Red/Yellow/Green release decision.
- Actions: GitHub inline comments + commit-status gate, Discord alerts, SQLite persistence, weekly risk report.

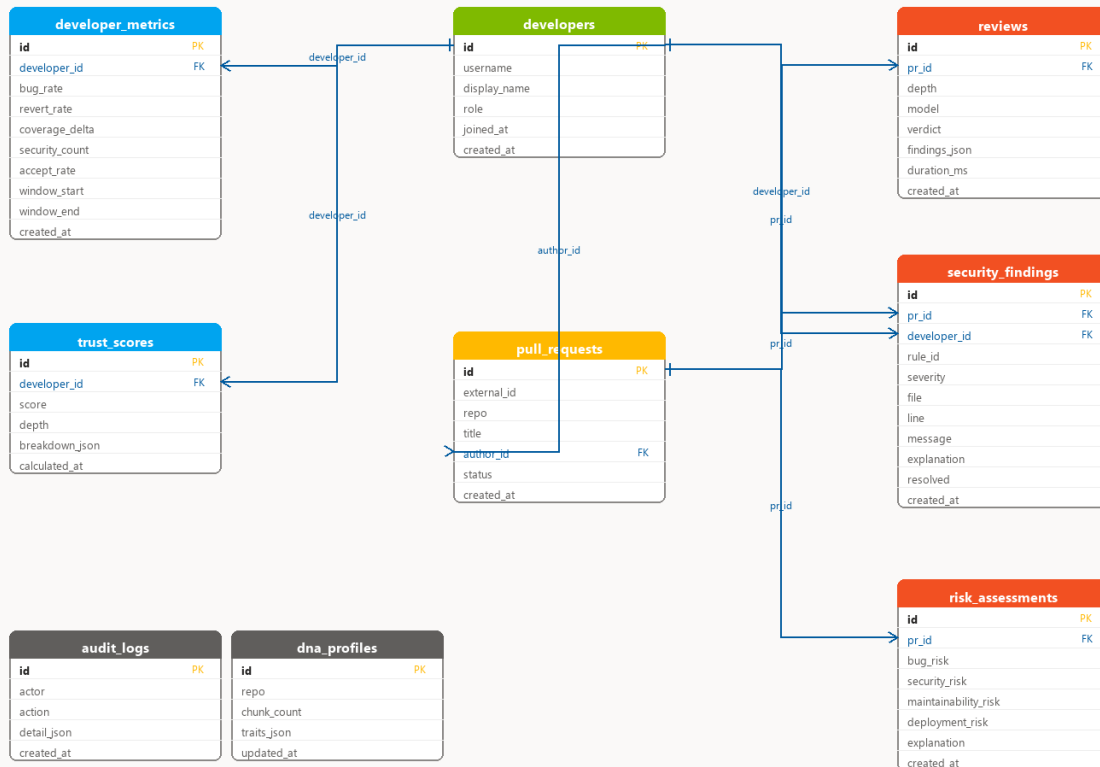
2. REST API

POST /webhook	GitHub PR webhook (HMAC-SHA256 verified)
GET /health	liveness + live/mock integration status
GET /dashboard	Tailwind SPA dashboard
GET /api/developers	developers + DTS + 12-week history
GET /api/reviews	recent AI reviews (repo, author, verdict)
GET /api/findings	security findings (repo / PR tagged)
GET /api/dna/profile	learned DNA traits for a repo
POST /api/dna/ingest	build DNA for a repo (fixtures or real GitHub code)
GET /api/report	weekly anonymised team risk report
GET /api/deploy-gate	Red/Yellow/Green release gate
GET /api/audit	audit-log entries
POST /api/demo/simulate	run full pipeline on a bundled diff (offline demo)

3. Data model

SQLite (WAL mode) with nine tables. Foreign keys link metrics, trust scores, pull requests, reviews, findings and risk assessments; dna_profiles and audit_logs are standalone. CHECK constraints guard the depth, status and resolved values.

DevGuardian — Database Schema (SQLite)



4. Multi-repository support

- Per-repo DNA profiles: each repository learns and is compared against its own history (ChromaDB filtered by repo).
- Ingest real repo code: POST /api/dna/ingest?repo=<r>&owner=<o>&ref=<branch> fetches up to 40 Python files via the GitHub API.
- Dashboard repo filter on the Security tab; every review and finding shows a repo badge + PR number.
- Onboarding at scale: a service (bot) account as collaborator, or a GitHub App installed on selected repos.

5. Real-time dashboard

- Auto-refreshes the review feed every 5s with a live indicator; pauses when the browser tab is backgrounded.
- Tabs: Trust Overview, Live PR Review (offline simulate), Security, Codebase DNA, Team Intelligence.
- Microsoft 365 Fluent theme; the four MS logo colours drive all charts and badges.

6. Security, deployment & testing

- Mock mode: a missing NVIDIA / GitHub / Discord key auto-degrades that integration; the full pipeline runs offline.
- Webhook HMAC verification refuses requests when no secret is configured.
- Deploy: Docker + docker-compose (seed-on-first-start entryptoint); local via ngrok for live GitHub webhooks.
- CI: GitHub Actions + Azure Pipelines run pytest, the seeder, and a Semgrep self-scan.
- Tests: 28 pytest tests (DTS formula, security rules, webhook + full demo pipeline).

7. Pull-request review flow

